the elements of S that is good for searching. Suppose that you have a probability distribution, where $p(x_i)$ is the probability that a given search searches for element x_i. Argue that the expected cost for m searches is

$$m \sum_{i=1}^{n} p(x_i) \cdot r_L(x_i) .$$

Prove that this sum is minimized when the elements of L are sorted in decreasing order with respect to $p(x_i)$.

27.2-2

Professor Carnac claims that since FORESEE is an optimal algorithm that knows the future, then at each step it must incur no more cost than MOVE-TO-FRONT. Either prove that Professor Carnac is correct or provide a counterexample.

27.2-3

Another way to maintain a linked list for efficient searching is for each element to maintain a ***frequency count***: the number of times that the element has been searched for. The idea is to rearrange list elements after searches so that the list is always sorted by decreasing frequency count, from largest to smallest. Either show that this algorithm is $O(1)$-competitive, or prove that it is not.

27.2-4

The model in this section charged a cost of 1 for each swap. We can consider an alternative cost model in which, after accessing x, you can move x anywhere earlier in the list, and there is no cost for doing so. The only cost is the cost of the actual accesses. Show that MOVE-TO-FRONT is 2-competitive in this cost model, assuming that the number requests is sufficiently large. (*Hint:* Use the potential function $\Phi_i = I(L_i^M, L_i^F)$.)

27.3 Online caching

In Section 15.4, we studied the caching problem, in which ***blocks*** of data from the main memory of a computer are stored in the ***cache***: a small but faster memory. In that section, we studied the offline version of the problem, in which we assumed that we knew the sequence of memory requests in advance, and we designed an algorithm to minimize the number of cache misses. In almost all computer systems, caching is, in fact, an online problem. We do not generally know the series of cache requests in advance; they are presented to the algorithm only as the requests for blocks are actually made. To gain a better understanding of this more realistic scenario, we analyze online algorithms for caching. We will first see that all deterministic online algorithms for caching have a lower bound of $\Omega(k)$ for the competitive ratio, where k is the size of the cache. We will then present an algorithm with a competitive ratio of $\Theta(n)$, where the input size is n, and one with a competitive ratio of $O(k)$, which matches the lower bound. We will end by showing how to use randomization to design an algorithm with a much better competitive ratio of $\Theta(\lg k)$. We will also discuss the assumptions that underlie randomized online algorithms, via the notion of an adversary, such as we saw in Chapter 11 and will see in Chapter 31.

You can find the terminology used to describe the caching problem in Section 15.4, which you might wish to review before proceeding.

27.3.1 Deterministic caching algorithms

In the caching problem, the input comprises a sequence of n memory requests, for data in blocks $b_1, b_2, \ldots, b_n$, in that order. The blocks requested are not necessarily distinct: each block may appear multiple times within the request sequence. After block b_i is requested, it resides in a cache that can hold up to k blocks, where k is a fixed cache size. We assume that $n > k$, since otherwise we are assured that the cache can hold all the requested blocks at once. When a block b_i is requested, if it is already in the cache, then a ***cache hit*** occurs and the cache remains unchanged. If b_i is not in the cache, then a ***cache miss*** occurs. If the cache contains fewer than k blocks upon a cache miss, block b_i is placed into the cache, which now contains one block more than before. If a

cache miss occurs with an already full cache, however, some block must be evicted from the cache before b_i can enter. Thus, a caching algorithm must decide which block to evict from the cache upon a cache miss when the cache is full. The goal is to minimize the number of cache misses over the entire request sequence. The caching algorithms considered in this chapter differ only in which block they decide to evict upon a cache miss. We do not consider abilities such as prefetching, in which a block is brought into the cache before an upcoming request in order to avert a future cache miss.

There are many online caching policies to determine which block to evict, including the following:

- First-in, first-out (FIFO): evict the block that has been in the cache the longest time.
- Last-in, first-out (LIFO): evict the block that has been in the cache the shortest time.
- Least Recently Used (LRU): evict the block whose last use is furthest in the past.
- Least Frequently Used (LFU): evict the block that has been accessed the fewest times, breaking ties by choosing the block that has been in the cache the longest.

To analyze these algorithms, we assume that the cache starts out empty, so that no evictions occur during the first k requests. We wish to compare the performance of an online algorithm to an optimal offline algorithm that knows the future requests. As we will soon see, all these deterministic online algorithms have a lower bound of $\Omega(k)$ for their competitive ratio. Some deterministic algorithms also have a competitive ratio with an $O(k)$ upper bound, but some other deterministic algorithms are considerably worse, having a competitive ratio of $\Theta(n/k)$.

We now proceed to analyze the LIFO and LRU policies. In addition to assuming that $n > k$, we will assume that at least k distinct blocks are requested. Otherwise, the cache never fills up and no blocks are evicted,

so that all algorithms exhibit the same behavior. We begin by showing that LIFO has a large competitive ratio.

Theorem 27.2
LIFO has a competitive ratio of $\Theta(n/k)$ for the online caching problem with n requests and a cache of size k.

Proof We first show a lower bound of $\Omega(n/k)$. Suppose that the input consists of $k + 1$ blocks, numbered $1, 2, \ldots, k + 1$, and the request sequence is

$$1, 2, 3, 4, \ldots, k, k + 1, k, k + 1, k, k + 1, \ldots,$$

where after the initial $1, 2, \ldots, k, k + 1$, the remainder of the sequence alternates between k and $k + 1$, with a total of n requests. The sequence ends on block k if n and k are either both even or both odd, and otherwise, the sequence ends on block $k+1$. That is, $b_i = i$ for $i = 1, 2, \ldots k-1$, $b_i = k+1$ for $i = k+1, k+3, \ldots$ and $b_i = k$ for $i = k, k + 2, \ldots$. How many blocks does LIFO evict? After the first k requests (which are considered to be cache misses), the cache is filled with blocks $1, 2, \ldots, k$. The $(k + 1)$st request, which is for block $k + 1$, causes block k to be evicted. The $(k + 2)$nd request, which is for block k, forces block $k + 1$ to be evicted, since that block was just placed into the cache. This behavior continues, alternately evicting blocks k and $k+1$ for the remaining requests. LIFO, therefore, suffers a cache miss on every one of the n requests.

The optimal offline algorithm knows the entire sequence of requests in advance. Upon the first request of block $k + 1$, it just evicts any block except block k, and then it never evicts another block. Thus, the optimal offline algorithm evicts only once. Since the first k requests are considered cache misses, the total number of cache misses is $k + 1$. The competitive ratio, therefore, is $n/(k + 1)$, or $\Omega(n/k)$.

For the upper bound, observe that on any input of size n, any caching algorithm incurs at most n cache misses. Because the input contains at least k distinct blocks, any caching algorithm, including the optimal offline algorithm, must incur at least k cache misses. Therefore, LIFO has a competitive ratio of $O(n/k)$.

■

We call such a competitive ratio ***unbounded***, because it grows with the input size. Exercise 27.3-2 asks you to show that LFU also has an unbounded competitive ratio.

FIFO and LRU have a much better competitive ratio of $\Theta(k)$. There is a big difference between competitive ratios of $\Theta(n/k)$ and $\Theta(k)$. The cache size k is independent of the input sequence and does not grow as more requests arrive over time. A competitive ratio that depends on n, on the other hand, does grow with the size of the input sequence and thus can get quite large. It is preferable to use an algorithm with a competitive ratio that does not grow with the input sequence's size, when possible.

We now show that LRU has a competitive ratio of $\Theta(k)$, first showing the upper bound.

Theorem 27.3
LRU has a competitive ratio of $O(k)$ for the online caching problem with n requests and a cache of size k.

Proof To analyze LRU, we will divide the sequence of requests into ***epochs***. Epoch 1 begins with the first request. Epoch i, for $i > 1$, begins upon encountering the $(k + 1)$st distinct request since the beginning of epoch $i - 1$. Consider the following example of requests with $k = 3$:

$$1, 2, 1, 5, 4, 4, 1, 2, 4, 2, 3, 4, 5, 2, 2, 1, 2, 2\,. \tag{27.10}$$

The first $k = 3$ distinct requests are for blocks 1, 2 and 5, so epoch 2 begins with the first request for block 4. In epoch 2, the first 3 distinct requests are for blocks 4, 1, and 2. Requests for these blocks recur until the request for block 3, and with this request epoch 3 begins. Thus, this example has four epochs:

$$1, 2, 1, 5 \qquad 4, 4, 1, 2, 4, 2 \qquad 3, 4, 5 \qquad 2, 2, 1, 2, 2\,. \tag{27.11}$$

Now we consider the behavior of LRU. In each epoch, the first time a request for a particular block appears, it may cause a cache miss, but subsequent requests for that block within the epoch cannot cause a cache miss, since the block is now one of the k most recently used. For

example, in epoch 2, the first request for block 4 causes a cache miss, but the subsequent requests for block 4 do not. (Exercise 27.3-1 asks you to show the contents of the cache after each request.) In epoch 3, requests for blocks 3 and 5 cause cache misses, but the request for block 4 does not, because it was recently accessed in epoch 2. Since only the first request for a block within an epoch can cause a cache miss and the cache holds k blocks, each epoch incurs at most k cache misses.

Now consider the behavior of the optimal algorithm. The first request in each epoch must cause a cache miss, even for an optimal algorithm. The miss occurs because, by the definition of an epoch, there *must* have been k other blocks accessed since the last access to this block.

Since, for each epoch, the optimal algorithm incurs at least one miss and LRU incurs at most k, the competitive ratio is at most $k/1 = O(k)$.

■

Exercise 27.3-3 asks you to show that FIFO also has a competitive ratio of $O(k)$.

We could show lower bounds of $\Omega(k)$ on LRU and FIFO, but in fact, we can make a much stronger statement: *any* deterministic online caching algorithm must have a competitive ratio of $\Omega(k)$. The proof relies on an adversary who knows the online algorithm being used and can tailor the future requests to cause the online algorithm to incur more cache misses than the optimal offline algorithm.

Consider a scenario in which the cache has size k and the set of possible blocks to request is $\{1, 2, \ldots, k + 1\}$. The first k requests are for blocks $1, 2, \ldots, k$, so that both the adversary and the deterministic online algorithm place these blocks into the cache. The next request is for block $k + 1$. In order to make room in the cache for block $k + 1$, the online algorithm evicts some block b_1 from the cache. The adversary, knowing that the online algorithm has just evicted block b_1, makes the next request be for b_1, so that the online algorithm must evict some other block b_2 to clear room in the cache for b_1. As you might have guessed, the adversary makes the next request be for block b_2, so that the online algorithm evicts some other block b_3 to make room for b_2.

The online algorithm and the adversary continue in this manner. The online algorithm incurs a cache miss on every request and therefore incurs n cache misses over the n requests.

Now let's consider an optimal offline algorithm, which knows the future. As discussed in Section 15.4, this algorithm is known as furthest-in-future, and it always evicts the block whose next request is furthest in the future. Since there are only $k + 1$ unique blocks, when furthest-in-future evicts a block, we know that it will not be accessed during at least the next k requests. Thus, after the first k cache misses, the optimal algorithm incurs a cache miss at most once every k requests. Therefore, the number of cache misses over n requests is at most $k + n/k$.

Since the deterministic online algorithm incurs n cache misses and the optimal offline algorithm incurs at most $k + n/k$ cache misses, the competitive ratio is at least

$$\frac{n}{k + n/k} = \frac{nk}{n + k^2}.$$

For $n \geq k^2$, the above expression is at least

$$\frac{nk}{n + k^2} \geq \frac{nk}{2n} = \frac{k}{2}.$$

Thus, for sufficiently long request sequences, we have shown the following:

Theorem 27.4

Any deterministic online algorithm for caching with a cache size of k has competitive ratio $\Omega(k)$.

■

Although we can analyze the common caching strategies from the point of view of competitive analysis, the results are somewhat unsatisfying. Yes, we can distinguish between algorithms with a competitive ratio of $\Theta(k)$ and those with unbounded competitive ratios. In the end, however, all of these competitive ratios are rather high. The online algorithms we have seen so far are deterministic, and it is this property that the adversary is able to exploit.

27.3.2 Randomized caching algorithms

If we don't limit ourselves to deterministic online algorithms, we can use randomization to develop an online caching algorithm with a significantly smaller competitive ratio. Before describing the algorithm, let's discuss randomization in online algorithms in general. Recall that we analyze online algorithms with respect to an adversary who knows the online algorithm and can design requests knowing the decisions made by the online algorithm. With randomization, we must ask whether the adversary also knows the random choices made by the online algorithm. An adversary who does not know the random choices is ***oblivious***, and an adversary who knows the random choices is ***nonoblivious***. Ideally, we prefer to design algorithms against a nonoblivious adversary, as this adversary is stronger than an oblivious one. Unfortunately, a nonoblivious adversary mitigates much of the power of randomness, as an adversary who knows the outcome of random choices typically can act as if the online algorithm is deterministic. The oblivious adversary, on the other hand, does not know the random choices of the online algorithm, and that is the adversary we typically use.

As a simple illustration of the difference between an oblivious and nonoblivious adversary, imagine that you are flipping a fair coin n times, and the adversary wants to know how many heads you flipped. A nonoblivious adversary knows, after each flip, whether the coin came up heads or tails, and hence knows how many heads you flipped. An oblivious adversary, on the other hand, knows only that you are flipping a fair coin n times. The oblivious adversary, therefore, can reason that the number of heads follows a binomial distribution, so that the expected number of heads is $n/2$ (by equation (C.41) on page 1199) and the variance is $n/4$ (by equation (C.44) on page 1200). But the oblivious adversary has no way of knowing exactly how many heads you actually flipped.

Let's return to caching. We'll start with a deterministic algorithm and then randomize it. The algorithm we'll use is an approximation of LRU called MARKING. Rather than "least recently used," think of MARKING as simply "recently used." MARKING maintains a 1-bit

attribute *mark* for each block in the cache. Initially, all blocks in the cache are unmarked. When a block is requested, if it is already in the cache, it is marked. If the request is a cache miss, MARKING checks to see whether there are any unmarked blocks in the cache. If all blocks are marked, then they are all changed to unmarked. Now, regardless of whether all blocks in the cache were marked when the request occurred, there is at least one unmarked block in the cache, and so an arbitrary unmarked block is evicted, and the requested block is placed into the cache and marked.

How should the block to evict from among the unmarked blocks in the cache be chosen? The procedure RANDOMIZED-MARKING on the next page shows the process when the block is chosen randomly. The procedure takes as input a block b being requested.

```
RANDOMIZED-MARKING(b)
1 if block b resides in the cache,
2     b.mark = 1
3 else
4     if all blocks b' in the cache have b'.mark = 1
5         unmark all blocks b' in the cache, setting b'.mark = 0
6     select an unmarked block u with u.mark = 0 uniformly at random
7     evict block u
8     place block b into the cache
9     b.mark = 1
```

For the purpose of analysis, we say that a new epoch begins immediately after each time line 5 executes. An epoch starts with no marked blocks in the cache. The first time a block is requested during an epoch, the number of marked blocks increases by 1, and any subsequent requests to that block do not change the number of marked blocks. Therefore, the number of marked blocks monotonically increases within an epoch. Under this view, epochs are the same as in the proof of Theorem 27.3: with a cache that holds k blocks, an epoch comprises requests for k distinct blocks (possibly fewer for the final

epoch), and the next epoch begins upon a request for a block not in those k.

Because we are going to analyze a randomized algorithm, we will compute the expected competitive ratio. Recall that for an input I, we denote the solution value of an online algorithm A by $A(I)$ and the solution value of an optimal algorithm F by $F(I)$. Online algorithm A has an ***expected competitive ratio*** c if for all inputs I, we have

$$\mathrm{E}[A(I)] \le cF(I) , \tag{27.12}$$

where the expectation is taken over the random choices made by A.

Although the deterministic MARKING algorithm has a competitive ratio of $\Theta(k)$ (Theorem 27.4 provides the lower bound and see Exercise 27.3-4 for the upper bound), RANDOMIZED-MARKING has a much smaller expected competitive ratio, namely $O(\lg k)$. The key to the improved competitive ratio is that the adversary cannot always make a request for a block that is not in the cache, since an oblivious adversary does not know which blocks are in the cache.

Theorem 27.5
RANDOMIZED-MARKING has an expected competitive ratio of $O(\lg k)$ for the online caching problem with n requests and a cache of size k, against an oblivious adversary.

Before proving Theorem 27.5, we prove a basic probabilistic fact.

Lemma 27.6
Suppose that a bag contains $x + y$ balls: $x - 1$ blue balls, y white balls, and 1 red ball. You repeatedly choose a ball at random and remove it from the bag until you have chosen a total of m balls that are either blue or red, where $m \le x$. You set aside each white ball you choose. Then, one of the balls chosen is the red ball with probability m/x.

Proof Choosing a white ball does not affect how many blue or red balls are chosen in any way. Therefore, we can continue the analysis as if there were no white balls and the bag contains just $x - 1$ blue balls and 1 red ball.

Let A be the event that the red ball is not chosen, and let A_i be the event that the ith draw does not choose the red ball. By equation (C.22) on page 1190, we have

$$\begin{aligned}\Pr\{A\} &= \Pr\{A_1 \cap A_2 \cap \cdots \cap A_m\} \\ &= \Pr\{A_1\} \cdot \Pr\{A_2 \mid A_1\} \cdot \Pr\{A_3 \mid A_1 \cap A_2\} \cdots \\ &\quad \Pr\{A_m \mid A_1 \cap A_2 \cap \cdots \cap A_{m-1}\} \; . \end{aligned} \tag{27.13}$$

The probability $\Pr\{A_1\}$ that the first ball is blue equals $(x - 1)/x$, since initially there are $x - 1$ blue balls and 1 red ball. More generally, we have

$$\Pr\{A_i \mid A_1 \cap \cdots \cap A_{i-1}\} = \frac{x - i}{x - i + 1} \; , \tag{27.14}$$

since the ith draw is from $x - i$ blue balls and 1 red ball. Equations (27.13) and (27.14) give

$$\Pr\{A\} = \left(\frac{x-1}{x}\right)\left(\frac{x-2}{x-1}\right)\left(\frac{x-3}{x-2}\right)\cdots\left(\frac{x-m+1}{x-m+2}\right)\left(\frac{x-m}{x-m+1}\right) \; . \tag{27.15}$$

The right-hand side of equation (27.15) is a telescoping product, similar to the telescoping series in equation (A.12) on page 1143. The numerator of one term equals the denominator of the next, so that everything except the first denominator and last numerator cancel, and we obtain $\Pr\{A\} = (x - m)/x$. Since we actually want to compute $\Pr\{\bar{A}\} = 1 - \Pr\{A\}$, that is, the probability that the red ball *is* chosen, we get $\Pr\{\bar{A}\} = 1 - (x - m)/x = m/x$.

■

Now we can prove Theorem 27.5.

Proof We'll analyze RANDOMIZED-MARKING one epoch at a time. Within epoch i, any request for a block b that is not the first request for block b in epoch i must result in a cache hit, since after the first request in epoch i, block b resides in the cache and is marked, so that it cannot be evicted during the epoch. Therefore, since we are counting cache misses, we'll consider only the first request for each block within each epoch, disregarding all other requests.

We can classify the requests in an epoch as either old or new. If block b resides in the cache at the start of epoch i, each request for block b during epoch i is an ***old request***. Old requests in epoch i are for blocks requested in epoch $i - 1$. If a request in epoch i is not old, it is a ***new request***, and it is for a block not requested in epoch $i - 1$. All requests in epoch 1 are new. For example, let's look again at the request sequence in example (27.11):

1, 2, 1, 5 4, 4, 1, 2, 4, 2 3, 4, 5 2, 2, 1, 2, 2.

Since we can disregard all requests for a block within an epoch other than the first request, to analyze the cache behavior, we can view this request sequence as just

1, 2, 5 4, 1, 2 3, 4, 5 2, 1.

All three requests in epoch 1 are new. In epoch 2, the requests for blocks 1 and 2 are old, but the request for block 4 is new. In epoch 3, the request for block 4 is old, and the requests for blocks 3 and 5 are new. Both requests in epoch 4 are new.

Within an epoch, each new request must cause a cache miss since, by definition, the block is not already in the cache. An old request, on the other hand, may or may not cause a cache miss. The old block is in the cache at the beginning of the epoch, but other requests might cause it to be evicted. Returning to our example, in epoch 2, the request for block 4 must cause a cache miss, as this request is new. The request for block 1, which is old, may or may not cause a cache miss. If block 1 was evicted when block 4 was requested, then a cache miss occurs and block 1 must be brought back into the cache. If instead block 1 was not evicted when block 4 was requested, then the request for block 1 results in a cache hit. The request for block 2 could incur a cache miss under two scenarios. One is if block 2 was evicted when block 4 was requested. The other is if block 1 was evicted when block 4 was requested, and then block 2 was evicted when block 1 was requested. We see that, within an epoch, each ensuing old request has an increasing chance of causing a cache miss.

Because we consider only the first request for each block within an epoch, we assume that each epoch contains exactly k requests, and each request within an epoch is for a unique block. (The last epoch might contain fewer than k requests. If it does, just add dummy requests to fill it out to k requests.) In epoch i, denote the number of new requests by $r_i \geq 1$ (an epoch must contain at least one new request), so that the number of old requests is $k - r_i$. As mentioned above, a new request always incurs a cache miss.

Let us now focus on an arbitrary epoch i to obtain a bound on the expected number of cache misses within that epoch. In particular, let's think about the jth old request within the epoch, where $1 \leq j < k$. Denote by b_{ij} the block requested in the jth old request of epoch i, and denote by n_{ij} and o_{ij} the number of new and old requests, respectively, that occur within epoch i but before the jth old request. Because $j - 1$ old requests occur before the jth old request, we have $o_{ij} = j - 1$. We will show that the probability of a cache miss upon the j th old request is $n_{ij}/(k - o_{ij})$, or $n_{ij}/(k - j + 1)$.

Start by considering the first old request, for block $b_{i,1}$. What is the probability that this request causes a cache miss? It causes a cache miss precisely when one of the $n_{i,1}$ previous requests resulted in $b_{i,1}$ being evicted. We can determine the probability that $b_{i,1}$ was chosen for eviction by using Lemma 27.6: consider the k blocks in the cache to be k balls, with block $b_{i,1}$ as the red ball, the other $k - 1$ blocks as the $k - 1$ blue balls, and no white balls. Each of the $n_{i,1}$ requests chooses a block to evict with equal probability, corresponding to drawing balls $n_{i,1}$ times. Thus, we can apply Lemma 27.6 with $x = k$, $y = 0$, and $m = n_{i,1}$, deriving the probability of a cache miss upon the first old request as $n_{i,1}/k$, which equals $n_{ij}/(k - j + 1)$ since $j = 1$.

In order to determine the probability of a cache miss for subsequent old requests, we'll need an additional observation. Let's consider the second old request, which is for block $b_{i,2}$. This request causes a cache miss precisely when one of the previous requests evicts $b_{i,2}$. Let's

consider two cases, based on the request for $b_{i,1}$. In the first case, suppose that the request for $b_{i,1}$ did not cause an eviction, because $b_{i,1}$ was already in the cache. Then, the only way that $b_{i,2}$ could have been evicted is by one of the $n_{i,2}$ new requests that precedes it. What is the probability that this eviction happens? There are $n_{i,2}$ chances for $b_{i,2}$ to be evicted, but we also know that there is one block in the cache, namely $b_{i,1}$, that is not evicted. Thus, we can again apply Lemma 27.6, but with $b_{i,1}$ as the white ball, $b_{i,2}$ as the red ball, the remaining blocks as the blue balls, and drawing balls $n_{i,2}$ times. Applying Lemma 27.6, with $x = k - 1$, $y = 1$, and $m = n_{i,2}$, we find that the probability of a cache miss is $n_{i,2}/(k - 1)$. In the second case, the request for $b_{i,1}$ does cause an eviction, which can happen only if one of the new requests preceding the request for $b_{i,1}$ evicts $b_{i,1}$. Then, the request for $b_{i,1}$ brings $b_{i,1}$ back into the cache and evicts some other block. In this case, we know that of the new requests, one of them did not result in $b_{i,2}$ being evicted, since $b_{i,1}$ was evicted. Therefore, $n_{i,2} - 1$ new requests could evict $b_{i,2}$, as could the request for $b_{i,1}$, so that the number of requests that could evict $b_{i,2}$ is $n_{i,2}$. Each such request evicts a block chosen from among $k - 1$ blocks, since the request that resulted in evicting $b_{i,1}$ did not also cause $b_{i,2}$ to be evicted. Therefore, we can apply Lemma 27.6, with $x = k - 1$, $y = 1$, and $m = n_{i,2}$, and get that the probability of a miss is $n_{i,2}/(k - 1)$. In both cases the probability is the same, and it equals $n_{ij}/(k - j + 1)$ since $j = 2$.

More generally, o_{ij} old requests occur before the jth old request. Each of these prior old requests either caused an eviction or did not. For those that caused an eviction, it is because they were evicted by a previous request, and for those that did not cause an eviction, it is because they were not evicted by any previous request. In either case, we can decrease the number of blocks that the random process is choosing from by 1 for each old request, and thus o_{ij} requests cannot cause b_{ij} to be evicted. Therefore, we can use Lemma 27.6 to determine

the probability that b_{ij} was evicted by a previous request, with $x = k - o_{ij}$, $y = o_{ij}$ and $m = n_{ij}$. Thus, we have proven our claim that the probability of a cache miss on the jth request for an old block is $n_{ij}/(k - o_{ij})$, or $n_{ij}/(k - j + 1)$. Since $n_{ij} \le r_i$ (recall that r_i is the number of new requests during epoch i), we have an upper bound of $r_i/(k - j + 1)$ on the probability that the jth old request incurs a cache miss.

We can now compute the expected number of misses during epoch i using indicator random variables, as introduced in Section 5.2. We define indicator random variables

$Y_{ij} = \mathrm{I}\{\text{the } j\text{th old request in epoch } i \text{ incurs a cache miss}\}$,

$Z_{ij} = \mathrm{I}\{\text{the } j \text{ th new request in epoch } i \text{ incurs a cache miss}\}$.

We have $Z_{ij} = 1$ for $j = 1, 2, \ldots, r_i$, since every new request results in a cache miss. Let X_i be the random variable denoting the number of cache misses during epoch i, so that

$$X_i = \sum_{j=1}^{k-r_i} Y_{ij} + \sum_{j=1}^{r_i} Z_{ij} ,$$

and so

$$\begin{aligned}
\mathrm{E}[X_i] &= \mathrm{E}\left[\sum_{j=1}^{k-r_i} Y_{ij} + \sum_{j=1}^{r_i} Z_{ij}\right] \\
&= \sum_{j=1}^{k-r_i} \mathrm{E}[Y_{ij}] + \sum_{j=1}^{r_i} \mathrm{E}[Z_{ij}] \quad \text{(by linearity of expectation)} \\
&\le \sum_{j=1}^{k-r_i} \frac{r_i}{k-j+1} + \sum_{j=1}^{r_i} 1 \quad \text{(by Lemma 5.1 on page 130)} \\
&= r_i \left(\sum_{j=1}^{k-r_i} \frac{1}{k-j+1} + 1\right) \\
&\le r_i \left(\sum_{j=1}^{k-1} \frac{1}{k-j+1} + 1\right) \\
&= r_i H_k \quad \text{(by equation (A.8) on page 1142)} , \qquad (27.16)
\end{aligned}$$

where H_k is the kth harmonic number.

To compute the expected total number of cache misses, we sum over all epochs. Let p denote the number of epochs and X be the random variable denoting the number of cache misses. Then, we have $X = \sum_{i=1}^{p} X_i$, so that

$$\begin{aligned}
\mathrm{E}[X] &= \mathrm{E}\left[\sum_{i=1}^{p} X_i\right] \\
&= \sum_{i=1}^{p} \mathrm{E}[X_i] && \text{(by linearity of expectation)} \\
&\le \sum_{i=1}^{p} r_i H_k && \text{(by inequality (27.16))} \\
&= H_k \sum_{i=1}^{p} r_i \,. && (27.17)
\end{aligned}$$

To complete the analysis, we need to understand the behavior of the optimal offline algorithm. It could make a completely different set of decisions from those made by RANDOMIZED-MARKING, and at any point its cache may look nothing like the cache of the randomized algorithm. Yet, we want to relate the number of cache misses of the optimal offline algorithm to the value in inequality (27.17), in order to have a competitive ratio that does not depend on $\sum_{i=1}^{p} r_i$. Focusing on individual epochs won't suffice. At the beginning of any epoch, the offline algorithm might have loaded the cache with exactly the blocks that will be requested in that epoch. Therefore, we cannot take any one epoch in isolation and claim that an offline algorithm must suffer any cache misses during that epoch.

If we consider two consecutive epochs, however, we can better analyze the optimal offline algorithm. Consider two consecutive epochs, $i-1$ and i. Each contains k requests for k different blocks. (Recall our assumption that all requests are first requests in an epoch.) Epoch i contains r_i requests for new blocks, that is, blocks that were not requested during epoch $i-1$. Therefore, the number of distinct requests during epochs $i-1$ and i is exactly $k+r_i$. No matter what the cache contents were at the beginning of epoch $i-1$, after $k + r_i$ distinct requests, there must be at least r_i cache misses. There could be more, but there is no way to have fewer. Letting m_i denote the number of

cache misses of the offline algorithm during epoch i, we have just argued that

$$m_{i-1} + m_i \geq r_i \ . \tag{27.18}$$

The total number of cache misses of the offline algorithm is

$$\begin{aligned}
\sum_{i=1}^{p} m_i &= \frac{1}{2}\sum_{i=1}^{p} 2m_i \\
&= \frac{1}{2}\left(m_1 + \sum_{i=2}^{p}(m_{i-1} + m_i) + m_p\right) \\
&\geq \frac{1}{2}\left(m_1 + \sum_{i=2}^{p}(m_{i-1} + m_i)\right) \\
&\geq \frac{1}{2}\left(m_1 + \sum_{i=2}^{p} r_i\right) && \text{(by inequality (27.18))} \\
&= \frac{1}{2}\sum_{i=1}^{p} r_i && \text{(because } m_1 = r_1\text{)} \ .
\end{aligned}$$

The justification $m_1 = r_1$ for the last equality follows because, by our assumptions, the cache starts out empty and every request incurs a cache miss in the first epoch, even for the optimal offline adversary.

To conclude the analysis, because we have an upper bound of $H_k \sum_{i=1}^{p} r_i$ on the expected number of cache misses for RANDOMIZED-MARKING and a lower bound of $\frac{1}{2}\sum_{i=1}^{p} r_i$ on the number of cache misses for the optimal offline algorithm, the expected competitive ratio is at most

$$\begin{aligned}
\frac{H_k \sum_{i=1}^{p} r_i}{\frac{1}{2}\sum_{i=1}^{p} r_i} &= 2H_k \\
&= 2\ln k + O(1) \quad \text{(by equation (A.9) on page 1142)} \\
&= O(\lg k) \ .
\end{aligned}$$

■

Exercises

27.3-1

For the cache sequence (27.10), show the contents of the cache after each request and count the number of cache misses. How many misses does each epoch incur?

27.3-2
Show that LFU has a competitive ratio of $\Theta(n/k)$ for the online caching problem with n requests and a cache of size k.

27.3-3
Show that FIFO has a competitive ratio of $O(k)$ for the online caching problem with n requests and a cache of size k.

27.3-4
Show that the deterministic MARKING algorithm has a competitive ratio of $O(k)$ for the online caching problem with n requests and a cache of size k.

27.3-5
Theorem 27.4 shows that any deterministic online algorithm for caching has a competitive ratio of $\Omega(k)$, where k is the cache size. One way in which an algorithm might be able to perform better is to have some ability to know what the next few requests will be. We say that an algorithm is ***l-lookahead*** if it has the ability to look ahead at the next l requests. Prove that for every constant $l \geq 0$ and every cache size $k \geq 1$, every deterministic l-lookahead algorithm has competitive ratio $\Omega(k)$.

Problems

27-1 Cow-path problem

The Appalachian Trail (AT) is a marked hiking trail in the eastern United States extending between Springer Mountain in Georgia and Mount Katahdin in Maine. The trail is about 2,190 miles long. You decide that you are going to hike the AT from Georgia to Maine and back. You plan to learn more about algorithms while on the trail, and so you bring along your copy of *Introduction to Algorithms* in your backpack.[2] You have already read through this chapter before starting out. Because the beauty of the trail distracts you, you forget about reading this book until you have reached Maine and hiked halfway back to Georgia. At that point, you decide that you have already seen the